Vision Foundation Models

STAT 578 - Statistical Foundations of SSL

Shulei Wang

University of Illinois at Urbana-Champaign

shuleiw@illinois.edu

https://sites.google.com/view/shulei-wang/



Where we start

If we hide part of an image,
what should the model be asked to predict?

In text the answer was handed to us: predict the missing *token*.
Images have no vocabulary, so we must choose.

Before we can ask that question properly,
we need to know what a vision model produces.

Overview

A Vision Transformer in Five Minutes

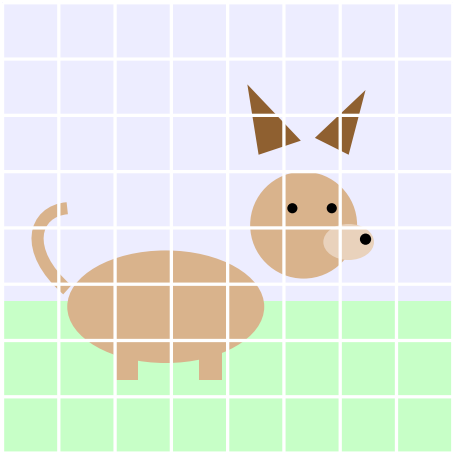
From BERT to MAE: Why Mask Images?

What Should the Target Be?

From SSL Method to Vision Foundation Model

Synthesis

From an image to a sequence



Cut the image into square patches:

$$\frac{224}{16} = 14, \qquad 14 \times 14 = 196.$$

- Each patch is **flattened** into a vector of pixel values.
- A linear map sends it to an embedding p_i.
- A **position encoding** is added so the model knows where the patch came from.

image $\longrightarrow$ **196 patches** $\longrightarrow$ **196 vectors** $p_1, \ldots, p_{196}$

What the Transformer does

$$(p_1, \ldots, p_{196}) \longrightarrow \text{Transformer} \longrightarrow (h_1, \ldots, h_{196})$$

- p_i — the input embedding of patch i, which knows only about that patch.
- h_i — the **contextual** representation of patch i, after repeatedly combining information from the other patches.

The vector for the ear patch can, after a few layers, depend on the grass, the tail, and the rest of the dog.

We will not need attention weights, heads or MLP blocks this week.

Two levels of representation

A ViT gives us representations at **two** levels, and the distinction matters for everything that follows.

Patch level (dense)

$$h_1, \ldots, h_{196}$$

one vector per location

Image level (global)

$$h_{\texttt{[CLS]}}$$

one vector for the whole image

The `[CLS]` token is an extra learned vector prepended to the sequence; it belongs to no patch, so it is free to summarise all of them.

That is all the ViT machinery we need:
representations at the patch level and at the image level.

Notation and colours for the week

Every method below fits one template:

$$X_{\text{vis}} \longrightarrow H \longrightarrow \text{pretext head} \longrightarrow \widehat{Y} \approx Y$$

X, X_{vis}	the image, and what the model is actually shown
H	the backbone representation — what we keep
Y	the self-supervised target
head	used during pretraining only, then discarded

The methods differ almost entirely in what Y is, and where it comes from.

Overview

A Vision Transformer in Five Minutes

From BERT to MAE: Why Mask Images?

What Should the Target Be?

From SSL Method to Vision Foundation Model

Synthesis

Can BERT's idea work on images?

We now have a sequence of tokens, exactly as in text. So the recipe seems to transfer: corrupt some patches, predict what was there.

	Text	**Image**
unit	word piece	16×16 patch
sequence length	a few hundred	196
representation	h_t per token	h_i per patch

The analogy is exact at the level of *architecture*.
It breaks at the level of the *target*.

Problem 1 — there is no vocabulary

In text the target lives in a finite set,

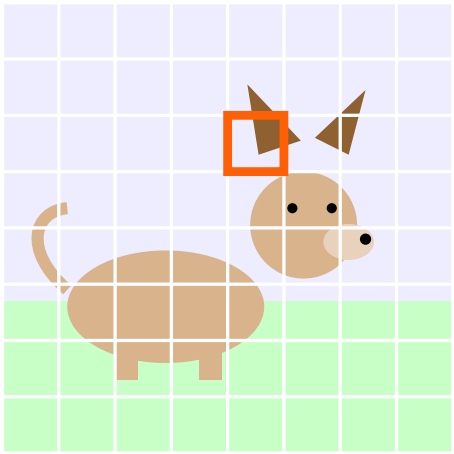
$$X_t \in \{1, \ldots, |\mathcal{V}|\},$$

so "predict the missing token" is a classification problem with a well-defined answer. A patch is a point in

$$\mathbb{R}^{16\times16\times3} = \mathbb{R}^{768}.$$

There is no natural discrete label for a patch, so there is no obvious thing to classify.

Problem 2 — spatial redundancy

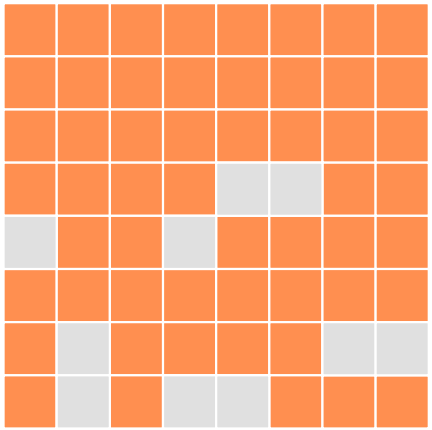


Hide the ear patch. Its neighbours are still visible — the other ear, the head, the sky above.

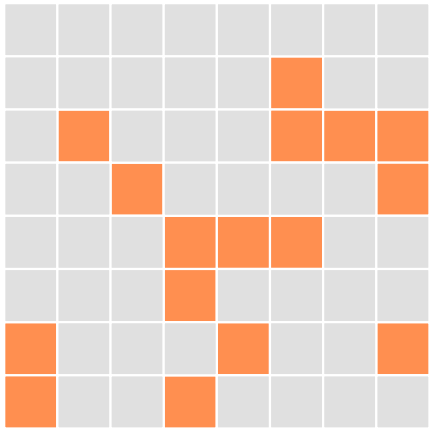
- Neighbouring patches are strongly correlated.
- A hidden patch of sky or grass can often be recovered by **interpolating** its neighbours.
- No understanding of the image is required.

Words are far less predictable from their immediate neighbours.
This is the difference that decides the design.

How much should we hide?



15% hidden



75% hidden (MAE)

visible hidden

At the text rate almost every hidden patch still has visible neighbours.

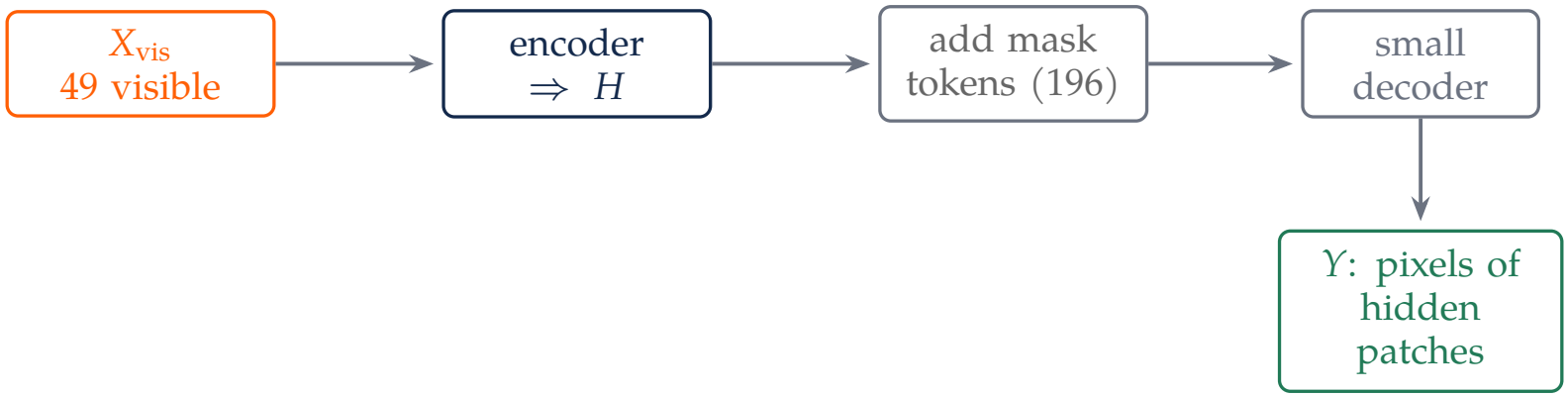
What raising the masking rate does

In language, raising the rate traded two quantities: more targets per sequence, but less context for each.

In images there is a *third* effect:
heavy masking removes the local interpolation shortcut.

- A high ratio greatly reduces the availability of adjacent visible patches, and so increases the pressure to use **broader image structure** rather than local continuation.
- The masking rate is therefore not only an efficiency knob — it changes *what information is useful* for solving the task.

MAE: the asymmetric design



1. The encoder sees **only visible patches**.
2. Mask tokens exist **only in the decoder**.

Target: pixels of the hidden patches
Produced by: the raw image itself
Keep: the ViT encoder H (the decoder is discarded)

Why the asymmetry matters

Computation

Attention cost grows quadratically in sequence length. Running the encoder on 49 rather than 196 patches cuts pretraining cost substantially, which is what makes very large models affordable.

Distribution shift

Because the encoder never processes a mask symbol, it is never trained on inputs containing a token that will not exist downstream — the mismatch that troubled masked language modelling does not arise.

Two problems addressed by a single architectural choice.

What squared-error prediction estimates

$$X_{\text{vis}} \longrightarrow H \longrightarrow \text{pretext head} \longrightarrow \widehat{Y} \approx Y = \text{pixels}$$

For the hidden ear patch, Y_i is its vector of RGB values, and

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|M|} \sum_{i \in M} \left\| \widehat{Y}_i - Y_i \right\|_2^2.$$

Squared error has a population minimiser we know exactly:

$$\widehat{Y}_i^{\star}(X_{\text{vis}}) \;=\; \mathbb{E}\big[\, Y_i \mid X_{\text{vis}} \,\big].$$

- The estimand is a **conditional mean of pixels**.
- Loss on visible patches is excluded — it would reward copying.

What the pixel target ignores

MAE works remarkably well: it scales to very large **ViT**s and transfers strongly, particularly after fine-tuning. The question is not whether it works.

Pixel reconstruction rewards *all* predictable pixel information,
including low-level appearance.
It does not explicitly privilege semantic information.

- Getting the ear's exact brown right and recognising that it *is* an ear both reduce the loss.
- The objective is indifferent between them.

What if we predicted a **more abstract** target?

What to remember

1. A **ViT** turns an image into patch tokens, and gives representations at two levels: per patch h_i, and per image $h_{\texttt{[CLS]}}$.
2. **High masking** removes the local interpolation shortcut that makes low-ratio masking too easy on images.
3. **MAE** predicts pixels with a cheap decoder; the **encoder is the representation**, and the decoder is thrown away.

Overview

A Vision Transformer in Five Minutes

From BERT to MAE: Why Mask Images?

What Should the Target Be?

From SSL Method to Vision Foundation Model

Synthesis

Three kinds of target

Option	Target Y	Where it comes from
raw data	pixels	the image itself
fixed abstraction	a code τ(patch)	a frozen tokenizer
learned abstraction	another network's output	a teacher that improves

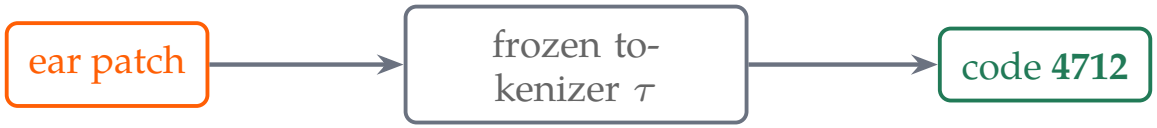
raw target $\longrightarrow$ fixed learned target $\longrightarrow$ online learned target

MAE is the first; **BEiT** the second; **DINO** and **iBOT** the third.

Option 2 — a frozen tokenizer

BEiT manufactures a visual vocabulary in advance. For this lecture you do *not* need to know how it is trained. Treat it as a frozen function

$$\tau : \text{ image patch } \longrightarrow \{1, \ldots, 8192\}.$$



$$\mathcal{L}_{\text{BEiT}} = -\sum_{i \in M} \log q_\theta\left(z_i \mid X_{\text{vis}}\right), \qquad z_i = \tau(\text{patch } i).$$

The tokenizer is trained separately — it exists only to construct targets.

Same architecture, new target space

$$X_{\text{vis}} \longrightarrow H \longrightarrow \text{pretext head} \longrightarrow \widehat{Y} \approx Y = \tau(\text{patch}) \in \{1, \ldots, 8192\}$$

	Target Y_i	**Loss**
MAE	pixels of the patch	squared error
BEiT	a visual token index	cross-entropy

BEiT also masks in contiguous **blocks** (about 40% of patches), so neighbours cannot fill the gap — the same concern that motivated a high ratio in **MAE**.

Target: a code $\tau(\text{patch}) \in \{1, \ldots, 8192\}$
Produced by: a frozen, separately trained tokenizer
Keep: the ViT encoder H

A tokenizer defines an equivalence

Discretisation is not the point. What matters is that the tokenizer decides **which distinctions the objective cares about**.

- If two patches receive the same code, $\tau(Y_i) = \tau(Y_i')$, the loss is *indifferent* to their pixel-level difference.
- Two slightly different browns on the ear may share a code — and then the objective stops asking about that difference.

Augmentation chooses what *input* variation to ignore.
Tokenization chooses what *target* variation to ignore.

But who trained the tokenizer?

The tokenizer is **external, frozen, and trained separately**, on its own data and its own objective.

- The equivalence classes that define the target were fixed before representation learning began.
- Nothing lets them adapt as the model's understanding improves.

Could the model manufacture its own targets, and improve them as it learns?

To answer that we leave reconstruction entirely, and return to joint embedding.

DINO leaves masking behind

Worth stating plainly, because the topic heading suggests otherwise.

- **DINO** does *not* mask patches.
- **DINO** does *not* reconstruct anything.
- It takes two augmented *crops* of the same image and asks two networks to agree.

DINO is much closer to BYOL than to MAE.

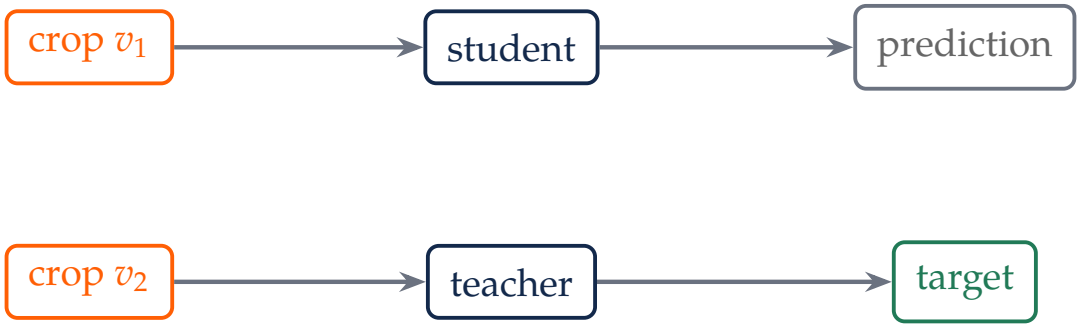
We are back in the joint-embedding family from earlier in the course.

Who supplies the target?

Take two augmented crops of the same image,

$$v_1 = A_1(X), \qquad v_2 = A_2(X),$$

and give them to two networks:



The student is trained. The teacher supplies the target. There are still no labels.

But where did the teacher come from?

The teacher is a slowly moving copy

"Teacher" invites the wrong picture. It is *not* a separately pretrained, superior model, and it has no external knowledge.

teacher = a slowly moving copy of the student

Why would that help?

- If the student had to match its *own current* output, both sides of the comparison would move at every step.
- A copy that changes slowly provides a target that holds still long enough to be chased.

Exactly the role the target network played in **BYOL**.

What is an EMA update?

Write θ_t for the student's parameters at step t and ξ_t for the teacher's. The teacher is never trained; it is *averaged*:

$$\boxed{\xi_t \;=\; m\,\xi_{t-1} \;+\; (1-m)\,\theta_t} \qquad m \text{ close to } 1.$$

With $m = 0.99$,

$$\xi_t = 0.99\,\xi_{t-1} + 0.01\,\theta_t :$$

the teacher moves only 1% of the way toward the current student at each update.

EMA averages network *parameters* over time.
It does not average images, and it does not average predictions.

One DINO training step

The target must also be treated as a constant while we differentiate — a **stop-gradient**, as in **BYOL**:

$$\ell \;=\; D\Big(\text{student}(v_1),\; \text{sg}\left\{\text{teacher}(v_2)\right\}\Big).$$

The loop

1. The student predicts the teacher's target.
2. Update the **student** by gradient descent.
3. Move the **teacher** slightly toward the student by EMA.

student: gradient descent teacher: EMA update

What does the teacher output?

So far we have not said *what* the two networks produce. The backbone gives H; **DINO** adds a small temporary network on top:

$$H \longrightarrow g(H) \in \mathbb{R}^K \longrightarrow \text{softmax} \longrightarrow p(\cdot \mid X).$$

- Think of the K coordinates as **learned output slots** — a few thousand of them. They are not ImageNet classes and carry no names.
- The **DINO** paper calls these coordinates *prototypes*; that is all the word means here.

The probability vector is the *pretraining target*.
The backbone representation H is what we *keep*.

The DINO objective

Teacher and student each produce a distribution over the K slots, and we match them with cross-entropy:

$$\mathcal{L} = -\sum_{k=1}^{K} p_{t,k}(v_2) \log p_{s,k}(v_1).$$

In words: the teacher says *"for this image, these are the slots I activate"*, and the student must produce the same pattern from a *different crop*.

Two words, defined

Distillation: a student learns from a teacher's predictions rather than from labels.

Self-distillation: the teacher itself comes from the student.

BYOL and DINO, side by side

	BYOL	DINO
student / teacher pair	yes	yes
EMA teacher	yes	yes
stop-gradient on the target	yes	yes
form of the target	a *vector*	a *distribution*

Only the last row is new.
DINO is not a fresh algorithm — it changes the form of the target.

And that change is what makes the next two slides possible.

What can collapse mean here?

Nothing so far stops teacher and student from agreeing on the same answer for *every* image. Take $K = 3$ and look at the teacher.

Same confident answer for everyone

$$p_t(X) \approx (0.99,\ 0.005,\ 0.005) \quad \text{for all } X$$

Same uncertain answer for everyone

$$p_t(X) = (1/3,\ 1/3,\ 1/3) \quad \text{for all } X$$

Both are the same disease:
the teacher's output has stopped carrying information about X.

Two problems, two fixes

Problem: all images activate the same slot

Fix: compare across the batch and suppress coordinates that are globally overused.

This is called **centering** — subtract a running mean from the teacher's logits.

Problem: each image produces a nearly uniform output

Fix: make each individual teacher target more decisive.

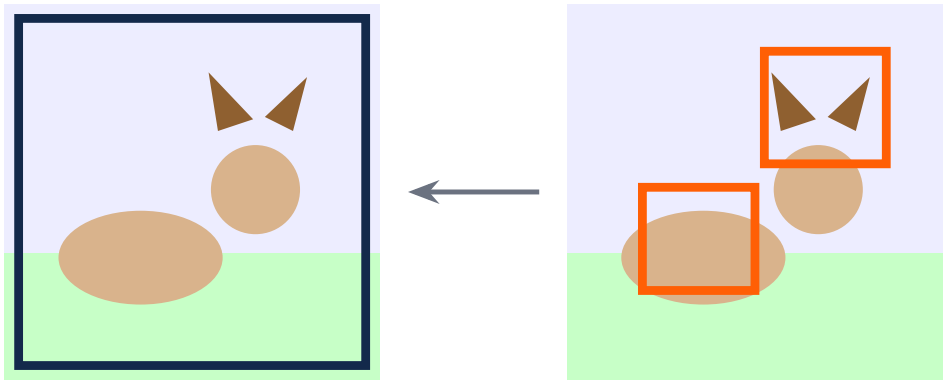
This is called **sharpening** — use a low temperature in the teacher's softmax.

centering acts *across* images sharpening acts *within* one image

This explains the mechanism. It is not a proof that collapse cannot occur.

Can we ask for more than agreement?

So far both crops could contain almost the same thing, and agreement would be easy. Can the prediction problem be made to demand more?



global crop → teacher

local crops → student

This is **multi-crop**: two large crops and several small ones, with the teacher seeing only the large ones.

What multi-crop asks for

A local crop is trained to produce an image-level target consistent with a global crop of the same image.

- So the objective rewards representations in which **local evidence is compatible with global image content** — something **MAE** never asks for.
- It is also where much of the compute efficiency comes from: local crops are small and cheap.
- It makes the emergent behaviour less surprising: `[CLS]` attention maps highlight object extent, and frozen features support strong ***k*-NN** classification.

Note this is a *consistency* requirement, not a guarantee that a crop of an ear determines the whole image.

Can the teacher supply patch targets?

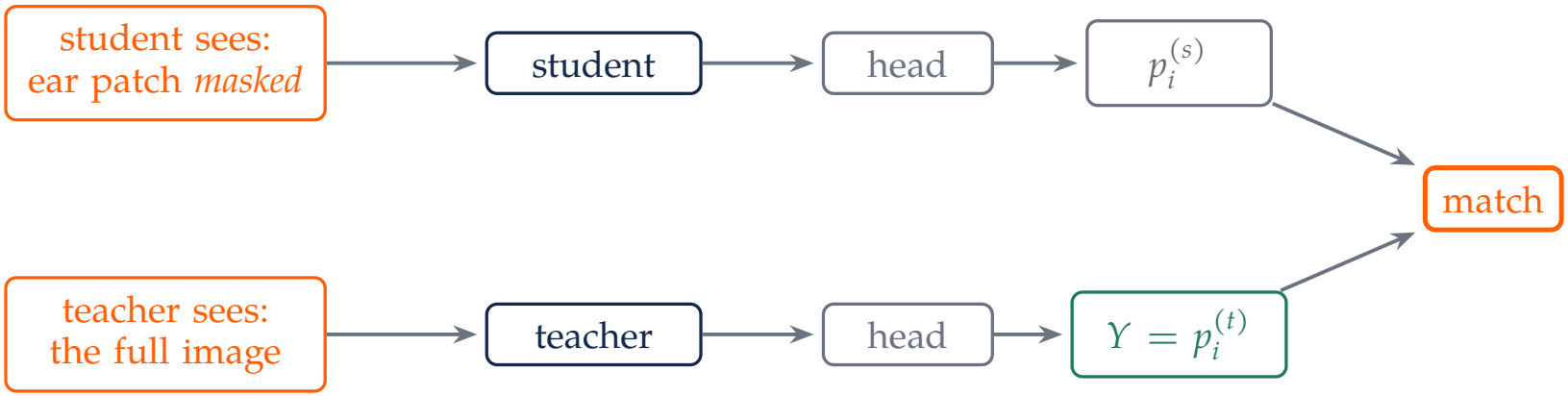
Two things we now have, each incomplete:

DINO	a learned target that improves — but only at the *image* level
BEiT	masked *patch* prediction — but against a frozen external target

Can we use DINO's teacher to create BEiT-like patch targets?

That is **iBOT**.

iBOT: the teacher becomes the tokenizer



The teacher tells the student what the missing patch should *mean*,
rather than what its RGB values should be.

Target: the teacher's patch output $p_i^{(t)}$, plus the image-level target
Produced by: the EMA teacher, online
Keep: the student backbone H

Two ways to read iBOT

BEiT	$\longrightarrow$	**iBOT**
offline tokenizer		online learned tokenizer
DINO	$+$	**iBOT**
global semantic signal		local masked-patch signal

Masked image modelling and self-distillation are not rivals.
iBOT is the first, with its target supplied by the second.

Plain English, then the technical name

The idea	The name
a slowly moving copy of the student	EMA teacher
do not backpropagate through the target	stop-gradient
a temporary output network, later discarded	projection / prediction head
K learned output slots with no names	prototypes
stop one slot dominating across images	centering
make each individual target decisive	sharpening
large and small views of the same image	multi-crop
the teacher creates patch targets as it learns	online tokenizer (iBOT)

Every row on the right is only a label for the row on its left.

Now the taxonomy: four design questions

With all four methods in hand, the differences sort into independent axes:

Method	Student sees	Target Y	Level	Target source
MAE	visible patches	pixels	patch	the image itself
BEiT	masked image	discrete code	patch	frozen tokenizer
DINO	augmented crop	teacher distribution	image	EMA teacher
iBOT	masked crop	teacher patch output + `[CLS]`	patch + image	EMA teacher
DINOv2	crops + masks	DINO + iBOT targets	image + patch	EMA teacher
DINOv3	crops + masks	the above + geometric anchor	image + patch	EMA + anchor

Read down the last column: the target source moves from the data, to a fixed external model, to the model itself.

What to remember

1. **BEiT** uses a **frozen tokenizer** to define abstract targets — and thereby decides which pixel differences the objective ignores.
2. **DINO** uses an **EMA teacher** to define learned global targets, with centering and sharpening against two distinct collapse modes.
3. **iBOT** combines teacher targets with patch masking: the teacher becomes an **online tokenizer**.

Overview

A Vision Transformer in Five Minutes

From BERT to MAE: Why Mask Images?

What Should the Target Be?

From SSL Method to Vision Foundation Model

Synthesis

Which one billion images?

Suppose the objective is settled — we have **DINO** plus **iBOT**, and it works.

If we are going to train on one billion web images,
***which* one billion images should we use?**

A good SSL loss is not enough.

This is **DINOv2**'s central claim, and the focus of this section.

What we are now asking for

One backbone, trained once, then *frozen* —
whose features can be reused for many visual tasks.

classification · retrieval · segmentation
depth estimation · correspondence

"Frozen" is the operative word: no fine-tuning, often only a linear layer or a nearest-neighbour lookup on top — and across image distributions the model was never tuned for.

Global and dense: different demands

Global: $h_{\texttt{[CLS]}}$

one vector per image

classification
retrieval

Dense: $h_1, \ldots, h_n$

one vector per patch

segmentation
depth, correspondence

A foundation model must be good at *both* at once.

Keep this distinction in view: the dense side is what will fail at scale.

DINOv2 invents no new pretext task

Its learning signal is, in words:

global DINO target + local iBOT target + feature spreading

- The first two we have already built.
- So if the objective is essentially known, the contribution must lie elsewhere.

So where can improvement come from?

$$\textbf{foundation representation} = \textbf{objective} + \textbf{data} + \textbf{scale and stability}$$

- We have the objective.
- The remaining two terms are the subject of this section — and the larger of them, and the focus of the reading, is the **data**.

More data is not automatically better

Suppose we crawl a million images and the pool happens to look like this:

visual mode	images
dogs	400,000
screenshots and memes	250,000
cars	150,000
birds	30,000
rare plants	5,000
everything else	...

Sample *images* uniformly and 40% of our gradient steps are spent on dogs.

no labels $\neq$ balanced data

If labels existed, balancing is easy

Suppose we knew a concept label for every image,

$$C(X) \in \{\text{dog}, \text{ car}, \text{ bird}, \ldots\}.$$

Then we could simply sample roughly equally across concepts,

$$P_{\text{train}}(C = c) \approx \frac{1}{K}.$$

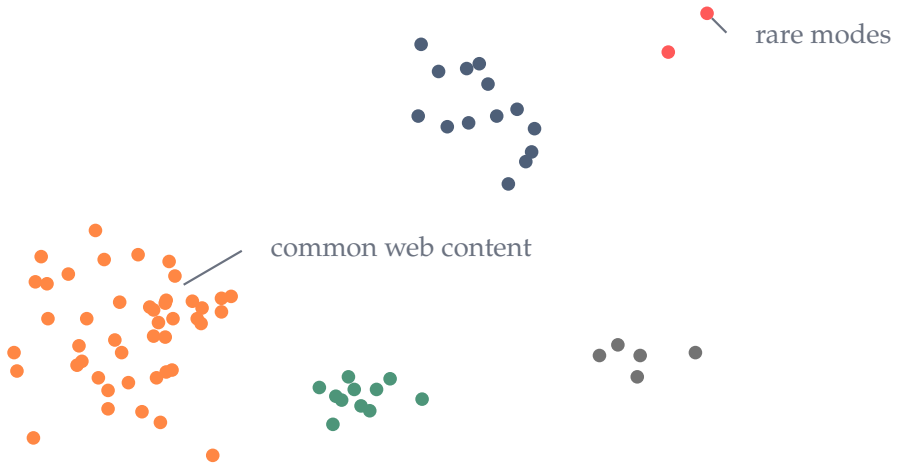
So in its simplest form, **data curation** means: change how often different kinds of observation appear during pretraining.

But self-supervised learning deliberately assumes those labels are unavailable.

With labels, imbalance is at least *visible*, and correctable by class-aware sampling. Without them we do not even know what the concepts are.

Without labels, we still have geometry

Embed every image with a representation we already trust: $X_i \mapsto h(X_i) \in \mathbb{R}^d$.



If the representation is any good, visually and semantically similar images occupy nearby regions: huge dense blobs for common content, tiny distant ones for rare modes.

We do not know the labels — but we do have the geometry.

Curation as choosing a distribution

Training on the raw crawl minimises one risk; training on a curated set minimises a *different* one:

$$R_P(\theta) = \mathbb{E}_{X \sim P_{\text{web}}}\big[\mathcal{L}_{\text{SSL}}(X;\theta)\big], \qquad R_Q(\theta) = \mathbb{E}_{X \sim Q}\big[\mathcal{L}_{\text{SSL}}(X;\theta)\big].$$

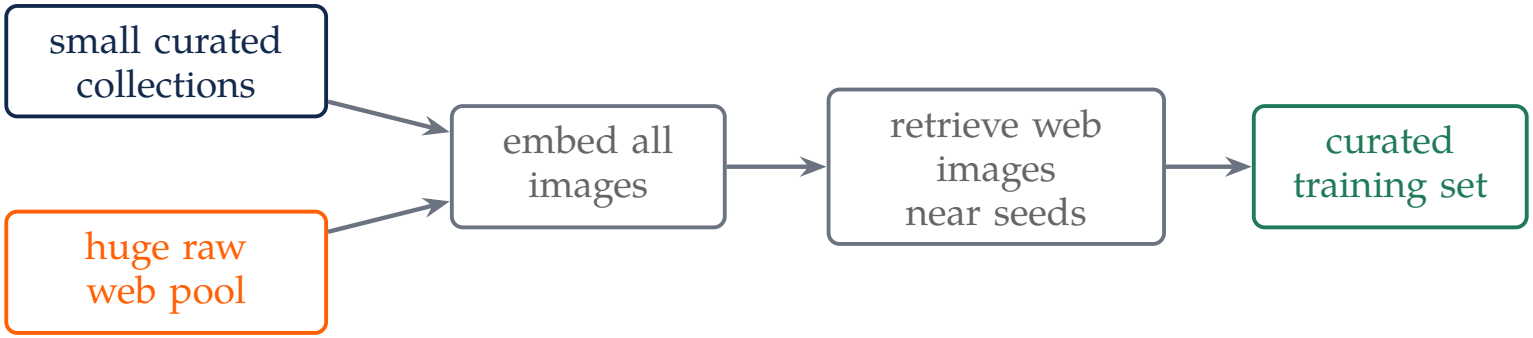
data curation = designing the training distribution Q

When the density ratio exists this is exactly reweighting:

$$R_Q(\theta) \;=\; \mathbb{E}_{P_{\text{web}}}\big[\, w(X)\, \mathcal{L}_{\text{SSL}}(X;\theta)\,\big], \qquad w = \frac{dQ}{dP_{\text{web}}}.$$

Selection, subsampling and reweighting are three ways of changing the same measure.

DINOv2: curated seeds plus retrieval



The curated collections say **which regions of representation space we want represented**; the web supplies volume around those regions.

Plus deduplication, which also removes anything resembling test or validation images.

The pipeline in detail: LVD-142M

1. **Sources.** A curated pool (ImageNet-22k, ImageNet-1k, Google Landmarks, fine-grained sets) and roughly 1.2B uncurated web images.
2. **Embed.** A self-supervised **ViT**-H/16 embeds every image; cosine similarity gives distances.
3. **Deduplicate.** Copy detection removes near-duplicates, and anything resembling test or validation images.
4. **Retrieve.** Cluster the uncurated pool; for each curated query image retrieve its nearest uncurated neighbours.
5. **Result.** 142M images.

But who chose the seeds?

The retrieval is only as good as the query set — and the query set is a hand-assembled list of datasets someone already considered interesting.

- A visual concept present in the web pool but absent from the seeds may never be retrieved.
- So the curation is **guided**: it scales a distribution we already chose, rather than discovering one.

Can curation be made automatic — inferred from the raw pool itself?

That is the question Vo et al. (2024) take up.

What makes a good SSL dataset?

Vo et al. propose three desiderata:

large + **diverse** + **balanced**

- **Large** and **diverse** come almost for free at web scale.
- **Balanced** is the hard one — because balancing normally requires knowing the categories, and here we do not.

Clusters as proxy concepts

Embed everything, then **cluster** in representation space.

- A **dense** region is, plausibly, a common visual mode.
- A **sparse** region is, plausibly, a rarer one.

Then sample more evenly *across regions* rather than uniformly across images.

Clusters are *not* semantic classes.
They are geometric proxies for them.

Everything downstream inherits whatever the embedding got right — or wrong.

Why *hierarchical* k-means?

One flat partition is a crude description of a billion heterogeneous images: a single k is either too coarse for the dense regions or too fine for the sparse ones.



Apply k-means repeatedly to build a hierarchy, then sample in a balanced way *down* that hierarchy — evenly among top-level clusters, then evenly within each, and so on.

No algorithmic details needed; the structure is the point.

What "balanced" means statistically

Raw web sampling draws $X \sim P$. The stated ideal is instead

$$X \sim U_{\mathrm{supp}(P)},$$

approximately **uniform over the support** of the data distribution — in the chosen embedding space.

- Dense regions are downweighted; sparse regions are relatively upweighted.
- Coverage of the support replaces fidelity to the original density as the goal.

uniform in representation space $\neq$ uniform over true concepts

Whether the two coincide depends entirely on whether the embedding organises the data meaningfully.

Does imbalance really matter?

A cleaner test than comparing web-scale pipelines: take a dataset that *is* balanced, and make it artificially long-tailed.

- Hold the visual universe fixed — the same classes, the same images available.
- Vary only the sampling distribution, increasing the imbalance.
- Reported ImageNet accuracy falls from about 82.7 in the balanced case to about 57.0 in the most imbalanced setting studied.

same visual universe + different sampling distribution
$\longrightarrow$ different representation quality

So the case for curation is empirical, not merely philosophical.

DINOv2 and Vo et al. side by side

	DINOv2	Vo et al.
Raw source	huge web pool	huge raw pool
Features used to select	SSL features	SSL features
How useful regions are found	curated seed datasets	clustering the raw pool
Selection rule	retrieve neighbours of seeds	hierarchical balanced sampling
Human-curated seed needed?	**yes**	**no**
Main idea	scale a chosen distribution	infer a balanced one

DINOv2 shows that curation matters.
Vo et al. make curation itself into an inference problem.

The remaining term: scale and stability

An objective that trains stably on ImageNet can become unstable, or prohibitively expensive, at billion-parameter scale.

DINOv2 adds two devices whose *roles* are worth knowing, even though the formulas are not:

- **KoLeo** — prevents representations from *crowding* together in the batch.
- **Sinkhorn–Knopp** — prevents *imbalance* in how the output slots are used.

Plus efficient attention, sequence packing and distributed training: necessary engineering, not conceptual content.

Optional: the two formulas

KoLeo — spread features apart

$$\mathcal{L}_{\text{KoLeo}} = -\frac{1}{n}\sum_{i=1}^{n} \log d_{n,i}, \qquad d_{n,i} = \min_{j \neq i} \|x_i - x_j\|$$

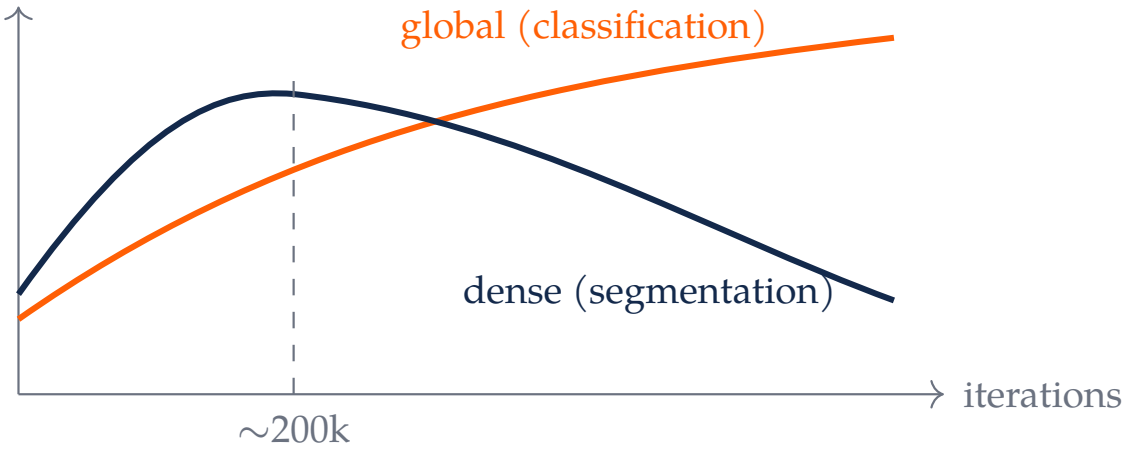
Penalise each feature for being close to its nearest neighbour — a differential-entropy estimator.

Sinkhorn–Knopp — balance slot usage

Normalise the batch of teacher assignments so the assignment matrix is approximately doubly stochastic, instead of relying on a running mean.

Neither formula needs to be memorised.

DINOv3: the two qualities diverge



Global representation quality can keep improving while local representation quality deteriorates.

The measured cause: cosine similarity between `[CLS]` and the patch outputs rises, so patch features lose locality. Judged by linear probing alone, nothing looks wrong.

Gram anchoring

Let P hold the ℓ_2-normalised patch features of one image. Its **Gram matrix**

$$G = PP^\top, \qquad G_{ij} = \cos(p_i, p_j),$$

records how similar every pair of patches is. Anchor it to an *earlier* checkpoint:

$$\mathcal{L}_{\text{Gram}} = \left\| G_{\text{student}} - G_{\text{anchor}} \right\|_F^2.$$

Why this is the right object

For any orthogonal R, $(PR)(PR)^\top = PRR^\top P^\top = PP^\top$.

The Gram matrix is **invariant to rotations of the feature space**, so anchoring preserves *pairwise relationships* rather than absolute coordinates — the representation stays free to evolve.

What to remember

1. **DINOv2** combines global and local self-supervised signals rather than inventing a new task.
2. Pretraining minimises $\mathbb{E}_{X \sim Q}[\mathcal{L}_{\text{SSL}}]$, so **curation is the design of** Q. **DINOv2** scales a chosen distribution; **Vo et al.** infer a balanced one by clustering.
3. At scale, **global and dense quality can diverge**; Gram anchoring repairs the dense side by preserving relationships, not coordinates.

Foundation-model design is partly loss design — but increasingly it is *distribution* design.

Overview

A Vision Transformer in Five Minutes

From BERT to MAE: Why Mask Images?

What Should the Target Be?

From SSL Method to Vision Foundation Model

Synthesis

The three transitions

MAE and BEiT

What should a masked patch predict? Pixels are intrinsic but indiscriminate; a tokenizer is selective but external and frozen.

DINO and iBOT

Can the model manufacture its own semantic targets? Yes — with an EMA teacher, and the collapse problem returns as the price.

DINOv2 and DINOv3

What else is required once the representation becomes infrastructure? A data distribution, scale, stability and dense quality — not only an objective.

What this week does not settle

- **No theory selects the target.** That learned targets help frozen-feature tasks is an empirical finding.
- **Centering and sharpening are not proved to prevent collapse.** They counter two degenerate patterns; that is a mechanism, not a guarantee.
- **The dense degradation is diagnosed, not explained.** We can measure `[CLS]`–patch similarity rising; no model predicts when it begins.
- **Gram anchoring is a repair** applied late, not a removal of the cause.
- **There is no universally correct pretraining distribution.** Uniform-over-support in a chosen embedding is a principled design choice, not a theorem that it is optimal for every downstream task.